

SAPIENZA
UNIVERSITÀ DI ROMA

Computer Vision

Appunti del corso

Facoltà di Ingegneria dell'Informazione, Informatica e Statistica
Corso di Laurea Magistrale in Computer Science

Docente
Prof. Marco Raoul Marini

Appunti di
Alessio Cavedal
GitHub: https://github.com/AlessioCavedal

Anno Accademico 2026/2027

Indice

1 Introduzione alla computer vision **1**
1.1 Il corso . . . 1
1.2 Che cos'è la computer vision . . . 1
1.3 Perché la visione è difficile . . . 3
1.4 Gli indizi visivi . . . 4
1.5 Visione, grafica e machine learning . . . 5
1.6 Breve storia . . . 6
1.7 Il programma del corso . . . 6
1.8 Applicazioni e ricerca . . . 8

2 Acquisizione e immagine digitale **10**
2.1 Dalla scena al sensore . . . 10
2.2 Il modello di formazione dell'immagine . . . 11
2.3 L'immagine digitale . . . 13
2.4 Campionamento . . . 14
2.5 Quantizzazione . . . 14
2.6 Rappresentazione e tipi di immagine . . . 16
2.7 Risoluzione spaziale . . . 17
2.8 Relazioni tra pixel . . . 18
2.9 Ridimensionamento e interpolazione . . . 20

3 Il colore **24**
3.1 Che cos'è il colore . . . 24
3.2 Luce e superfici . . . 25
3.3 L'occhio . . . 26
3.4 Percezione del colore e metameri . . . 27
3.5 Colorimetria . . . 28
3.6 Spazi colore . . . 30
3.7 Costanza e adattamento . . . 32
3.8 Bilanciamento del bianco . . . 34
3.9 Usi del colore nella computer vision . . . 35
3.10 Texture e Local Binary Pattern . . . 36
3.11 L'immagine in memoria . . . 37

Capitolo 1

Introduzione alla computer vision

1.1 Il corso

I testi di riferimento sono *Computer Vision: A Modern Approach* di D. Forsyth e J. Ponce (Prentice Hall) e *Computer Vision: Algorithms and Applications* di R. Szeliski (Springer), entrambi nella seconda edizione. Sono facoltativi *Introductory Techniques for 3-D Computer Vision* di E. Trucco e A. Verri (Prentice Hall, 1998) e *Digital Image Processing* di R. C. Gonzalez e R. E. Woods (Prentice Hall).

Gli strumenti di lavoro sono i linguaggi C/C++ e Python (o Matlab) e la libreria OpenCV.

1.2 Che cos'è la computer vision

Definizione (computer vision). La **computer vision** (visione artificiale) estrae da una o più immagini proprietà del mondo tridimensionale che le ha prodotte. In una formula: far vedere i calcolatori (*making computers see*).

Davanti a una scena di traffico, per esempio, le proprietà da estrarre possono essere il numero e il tipo dei veicoli, la posizione dell'ostacolo più vicino, il grado di congestione, il luogo in cui è stata ripresa la scena. Le ragioni per studiare la disciplina sono evidenti: immagini e video sono ovunque, dagli album fotografici personali ai film, dalle notizie allo sport, dalla videosorveglianza alle immagini mediche e scientifiche.

La Figura 1.1 mette a confronto la visione umana e quella artificiale. In entrambe un dispositivo di acquisizione (l'occhio, la telecamera) trasforma la scena in un segnale e un dispositivo di interpretazione (il cervello, il calcolatore) ne ricava delle interpretazioni. Davanti al dipinto di un giardino con un ponte sull'acqua, per esempio, parole come «giardino», «primavera», «ponte», «acqua».

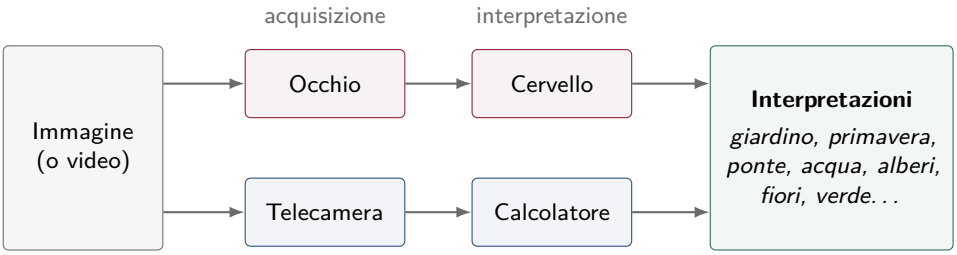


Figura 1.1. Visione umana (in alto) e artificiale (in basso): l'immagine viene acquisita da un sensore e interpretata da un sistema di elaborazione.

Misura e significato. La visione si usa in due modi. Come **strumento di misura** ha avuto grandi successi:

- la stereoscopia in tempo reale (*real-time stereo*);
- la ricostruzione della struttura 3D di una scena dal moto della camera (*structure from motion*);
- la ricostruzione 3D da collezioni di foto scattate da persone diverse (*multi-view stereo*);
- i rover della NASA su Marte.

Come **fonte di informazione semantica**, invece, l'immagine va capita. Dalla foto di una piazza con un edificio storico, per esempio, si possono ricavare tre tipi di informazione:

- la categoria degli oggetti (*object categorization*): cielo, edificio, bandiere, un volto su un ritratto, striscioni, lampioni, autobus, automobili, un muro;
- la categoria della scena e il contesto (*scene and context categorization*): all'aperto, in città, nel traffico;
- l'informazione spaziale qualitativa (*qualitative spatial information*): superfici verticali, orizzontali o inclinate, oggetti rigidi in movimento (gli autobus) e oggetti non rigidi (una persona).

Ci si può chiedere perché servano metodi di apprendimento, se l'obiettivo è solo far muovere un robot. Le risposte sono due:

- quella minore: per misurare, altri sensori sono spesso migliori delle telecamere, ma per navigare bisogna comunque imparare;
- quella maggiore: gli obiettivi della computer vision, cioè *che cosa* c'è nella scena e *dove* (*what + where*), sono definiti in base a ciò che interessa agli esseri umani.

Da qui viene anche il suo impatto sulla vita quotidiana: sicurezza (*safety* e *security*), salute, comfort, svago, accessibilità.

1.3 Perché la visione è difficile

Per una persona capire una foto è immediato. Si pensi a una foto scherzosa scattata in uno spogliatoio: un uomo si pesa su una bilancia mentre un altro, alle sue spalle, preme di nascosto la pedana con un piede, e i presenti ridono. Dove è stata scattata? Quante persone ci sono? Che cosa fanno? Su che cosa sta in piedi l'uomo? Perché la foto è divertente? Per rispondere, un sistema dovrebbe fare molte cose:

- ricostruire la geometria della stanza;
- accorgersi che alcune «persone» sono riflessi negli specchi;
- riconoscere la bilancia e notare il piede che la preme;
- riconoscere i volti;
- capire che chi si pesa non sa dello scherzo.

Servono cioè geometria, fisica e ragionamento sociale insieme. Non a caso il motto del corso è «un'immagine vale più di mille parole».

Un problema mal posto. Il mondo reale è molto più ricco di ciò che un'immagine ne misura. La formazione dell'immagine proietta la scena 3D su un piano 2D e in questo passaggio si perde informazione, a cominciare dalla profondità. Il processo, quindi, non si può invertire alla lettera: la visione è un **problema mal posto** (*ill-posed problem*).

Sfida	Che cosa cambia nell'immagine
Punto di vista e posa (*viewpoint*, *pose*)	lo stesso oggetto, visto da angolazioni diverse o in pose diverse, ha forme diverse (un volto scolpito fotografato da tre lati)
Illuminazione (*illumination*)	luci e ombre cambiano i valori dei pixel (lo stesso volto sotto luci diverse)
Scala (*scale*)	la dimensione nell'immagine non dice quella reale (lo stesso portatile tenuto da una persona molto alta e da una molto bassa)
Deformazione (*deformation*)	gli oggetti non rigidi cambiano forma (cavalli al galoppo)
Occlusione (*occlusion*)	parti dell'oggetto sono coperte da altri oggetti (un cavaliere tra gli alberi di un bosco)
Sfondo confuso (*background clutter*)	l'oggetto si mimetizza con lo sfondo (un granchio su un fondale con la sua stessa trama)
Variabilità intra-classe (*intra-class variation*)	oggetti della stessa categoria sono molto diversi tra loro (sedie di ogni forma)
Contesto (*context*)	in due immagini sfocate, la stessa macchia viene interpretata in modo diverso a seconda di ciò che la circonda

Tabella 1.1. Le principali sfide del riconoscimento visivo.

Parametri di disturbo. Lo stesso oggetto può produrre immagini molto diverse. I fattori che cambiano l'immagine senza cambiare ciò che c'è da riconoscere si chiamano **parametri di disturbo** (*nuisance parameters*): la Tabella 1.1 li riassume, insieme al ruolo del contesto.

Complessità. A tutto questo si aggiungono i numeri del problema:

- da migliaia a milioni di pixel per immagine;
- tra 3.000 e 30.000 categorie di oggetti che una persona sa riconoscere;
- oltre 30 gradi di libertà nella posa di un oggetto articolato come il corpo umano;
- la quantità di immagini, GIF e video sul web.

Non stupisce che nei primati circa metà della corteccia cerebrale sia dedicata all'elaborazione dell'informazione visiva.

1.4 Gli indizi visivi

Le immagini confondono, ma rivelano anche la struttura del mondo attraverso molti **indizi** (*cues*). Le sfide della sezione precedente sono quindi anche opportunità: il compito della computer vision è interpretare questi indizi. La Tabella 1.2 raccoglie i principali.

Indizio	Che cosa rivela	Come
Prospettiva lineare (*linear perspective*)	profondità	le linee parallele convergono e gli oggetti uguali rimpiccioliscono con la distanza (file di casette su un pontile)
Prospettiva aerea (*aerial perspective*)	profondità	gli oggetti lontani appaiono più chiari e meno contrastati (alberi in un bosco con la nebbia)
Occlusione	ordine in profondità	ciò che copre sta davanti (gli archi di un ponte, uno dentro l'altro)
Gradiente di texture (*texture gradient*)	forma	la trama si rimpicciolisce e si deforma con la distanza e l'inclinazione (un terreno secco e screpolato)
Ombreggiatura (*shading*)	forma e illuminazione	la luminosità cambia con l'orientazione della superficie (un torso scolpito)
Ombre portate (*cast shadows*)	posizione e illuminazione	l'ombra lega l'oggetto al suolo e indica da dove viene la luce (una sfera)
Somiglianza (*similarity*)	raggruppamento	elementi simili per colore, texture o vicinanza vanno insieme (fiori su un prato)
Destino comune (*common fate*)	raggruppamento	gli elementi che si muovono insieme vanno insieme (una carovana di cammelli con le loro ombre)

Tabella 1.2. Indizi di profondità, forma, illuminazione e raggruppamento.

1.5 Visione, grafica e machine learning

Problemi inversi. La grafica al calcolatore (*computer graphics*) parte da una descrizione della scena 3D (oggetti, materiali, forma, moto, posa) e produce un'immagine 2D: risolve un problema di **sintesi**. La visione fa il cammino opposto: dall'immagine, cioè da una matrice di pixel, risale alla scena. Il suo è un problema di **analisi**, inverso rispetto alla grafica (Figura 1.2), ed è la direzione difficile, perché la proiezione non si inverte.

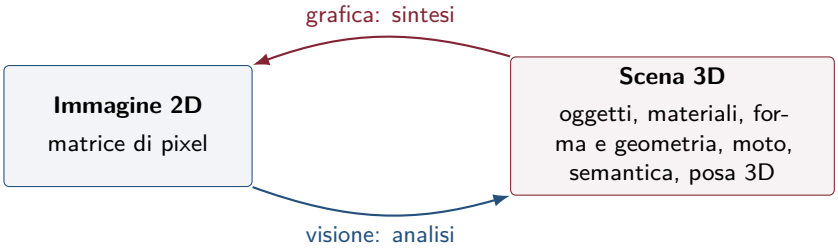


Figura 1.2. Grafica e visione come problemi inversi: la prima sintetizza l'immagine dalla scena, la seconda analizza l'immagine per risalire alla scena.

Riconoscimento di pattern. Molti compiti di visione hanno la forma di un problema di *pattern recognition* (riconoscimento di pattern). Un algoritmo statistico viene addestrato su esempi etichettati (fase di *training*) e poi, su un esempio nuovo (fase di *test*), ne indovina la classe. Tre famiglie di algoritmi:

- le *Support Vector Machine* (SVM), che portano i dati in uno spazio delle caratteristiche (*feature space*) dove le classi si separano;
- il classificatore *Gaussian Naive Bayes*;
- le reti neurali.

Un esempio curioso dello schema: le caratteristiche non vengono dalle immagini, ma dall'attività cerebrale di chi le guarda (i voxel di una risonanza magnetica). Dopo l'addestramento su foto di spiagge, l'algoritmo risponde «spiaggia» davanti a una foto nuova.

Computer vision e machine learning. Il *machine learning* (apprendimento automatico) studia algoritmi che permettono ai calcolatori di cambiare comportamento in base ai dati, per esempio quelli dei sensori o di un database. Il suo obiettivo principale è imparare a riconoscere pattern complessi e a prendere decisioni intelligenti a partire dai dati. È uno strumento utilissimo per la visione (*learning for vision*), ma la computer vision non si riduce all'apprendimento da esempi: comprende anche la modellazione, per esempio di come si forma un'immagine. Soprattutto, il machine learning di solito non si occupa di come si ottengono i dati; la computer vision sì. Deve decidere come acquisirli (il progetto dei sensori, la visione attiva) e come rappresentarli.

Le discipline vicine sono molte:

- elaborazione delle immagini (*image processing*);

- riconoscimento di pattern;
- fotogrammetria;
- grafica;
- intelligenza artificiale e machine learning;
- geometria proiettiva;
- teoria del controllo.

Più in largo, la computer vision si colloca all'incrocio tra due gruppi di discipline. Da un lato stanno informatica, ingegneria, matematica e fisica, dall'altro biologia e psicologia, con le neuroscienze e le scienze cognitive. Accanto ci sono la robotica, il riconoscimento del parlato e l'*information retrieval*.

1.6 Breve storia

La Tabella 1.3 riassume le tappe principali. L'episodio più citato è il *Summer Vision Project* del MIT. Nel 1966 Marvin Minsky affidò a uno studente del primo anno un problema da risolvere nel corso dell'estate: «collegare una telecamera a un calcolatore e fare in modo che la macchina descriva ciò che vede». Mezzo secolo dopo ci si sta ancora lavorando.

Periodo	Tappe
1963	Larry Roberts, considerato il padre della computer vision, nella tesi di dottorato al MIT ricostruisce il modello 3D di solidi poliedrici a partire da un'immagine (passando per un operatore di gradiente 2×2) e lo ridisegna da un punto di vista nuovo. Si interpretano mondi sintetici.
1966	*Summer Vision Project* al MIT.
Anni '70	Primi progressi su immagini selezionate: le *pictorial structures* di Fischler ed Elschlager (1973) descrivono un volto come insieme di parti (capelli, occhi, naso, bocca, contorni) legate da vincoli elastici.
Anni '80	Le reti neurali vanno e vengono; ci si sposta verso la geometria e un maggiore rigore matematico.
Anni '90	Riconoscimento dei volti; si affermano i metodi statistici.
Anni 2000	Riconoscimento più ampio, grandi dataset annotati, inizio dell'elaborazione dei video.
Anni 2010	Ritorno del deep learning: AlexNet (2012), DeepFace e DeepPose (2014), le didascalie automatiche di *Show, Attend and Tell* (2015).
Anni 2020	Veicoli autonomi.

Tabella 1.3. Tappe della computer vision.

1.7 Il programma del corso

Il corso è diviso nelle cinque parti della Tabella 1.4.

Parte	Argomenti
I. *Early vision*	formazione dell'immagine (camere, luce, colore) ed elaborazione di una singola immagine: filtri lineari, *edge detection* (rilevamento dei contorni), texture
II. *Mid-level vision*	raggruppamento e *fitting*: segmentazione di immagini e video, *tracking*, fitting ai minimi quadrati, allineamento
III. Geometria multi-vista	stereoscopia, *structure from motion*
IV. Riconoscimento	descrizione e *matching* di patch, *clustering* e vocabolari visivi, modelli *bag-of-features*, classificazione
V. Argomenti avanzati	visione di alto livello (posizione e orientazione di oggetti noti, contorni di superfici curve, *aspect graph*, dati di profondità), segmentazione, *face detection*, modelli articolati

Tabella 1.4. Le cinque parti del corso.

Tra i temi che torneranno ci sono:

- la geometria della formazione dell'immagine;
- i modelli di camera (a foro stenopeico, o *pinhole*, e con lenti), i loro parametri e la calibrazione;
- il filtraggio (convoluzione, *smoothing* o smussamento, *edge detection*, piramidi);
- la segmentazione in regioni e la texture;
- il restauro delle immagini e l'analisi della forma;
- la stereoscopia, il moto e il flusso ottico;
- il rilevamento e la semantica;
- i mosaici di immagini.

Lungo tutto il corso ricorrono anche due applicazioni: la ricerca di immagini in grandi collezioni e l'*image-based rendering*. Quest'ultimo crea immagini nuove a partire da immagini esistenti, perché costruire modelli che sembrino oggetti reali, con tutti i loro dettagli (come i segni dell'usura), è molto difficile.

Un'anticipazione: patch e texture. Nella prima parte servirà rappresentare piccole porzioni dell'immagine (*patch*), per tre ragioni:

- per stabilire corrispondenze tra punti di immagini diverse bisogna descriverne l'intorno;
- i cambiamenti bruschi di intensità, gli *edge*, sono importanti;
- la texture si descrive con statistiche sui tipi di patch presenti: le tigri hanno molte strisce e poche macchie, i leopardi il contrario.

Lo strumento è il **filtro**. Si appoggia un modello (*pattern*), cioè una piccola griglia di pesi, su una porzione dell'immagine, si moltiplica ogni peso per il pixel che gli

sta sotto e si sommano i prodotti: è il **prodotto scalare** tra il modello e quella porzione. Poi si sposta il modello e si ripete, su tutta l'immagine. Una risposta forte indica che, lì, l'immagine somiglia al modello. Un filtro fatto come una derivata, con una barra chiara accanto a una scura, misura le derivate dell'immagine. Con il modello $(-1, 0, 1)$, per esempio:

- tre pixel uniformi, $(10, 10, 10)$, danno $-10 + 0 + 10 = 0$: l'intensità non cambia;
- tre pixel in salita, $(10, 50, 90)$, danno $-10 + 0 + 90 = 80$: l'intensità cambia bruscamente, c'è un edge.

Anche la texture si rappresenta con statistiche delle risposte dei filtri. Per le tigri rispondono forte i filtri a barre su scala grossolana, per i ghepardi quelli a macchie della stessa scala, per l'erba quelli a barre lunghe e strette. Oggetti con texture diverse si possono così separare, e le variazioni della texture sono un indizio di forma (*shape from texture*).

Un problema tipo. Data un'immagine, contiene una persona? Le istanze positive variano moltissimo per posa, abbigliamento, scala e inquadratura; quelle negative vanno da animali a veicoli a paesaggi. È un buon esempio di quanto le sfide della Sezione 1.3 rendano difficile anche una domanda semplice.

1.8 Applicazioni e ricerca

Un successo emblematico è il rilevamento dei volti nelle fotocamere digitali: la camera trova i volti nella scena e su di essi regola in automatico la messa a fuoco (AF), l'esposizione (AE) e, se serve, il flash. La Tabella 1.5 raccoglie altri esempi, alcuni ormai storici.

La computer vision è un'area di ricerca molto attiva, che cambia rapidamente. Tra i temi di ricerca attivi ci sono:

- il riconoscimento di oggetti;
- l'analisi del comportamento umano;
- l'elaborazione di flussi video (*computer vision streams*);
- la biometria, compresa quella «soft»;
- la ricostruzione 3D su larga scala, fino a un'intera città;
- l'elaborazione di immagini mediche;
- la visione per la robotica.

Ambito	Esempi
Fotografia	rilevamento dei volti per messa a fuoco, esposizione e flash; scatto automatico quando il soggetto sorride
Documenti	riconoscimento ottico dei caratteri (OCR), dalle cifre scritte a mano alla lettura delle targhe
Biometria e sicurezza	riconoscimento del volto e dell'iride, lettori di impronte, accesso senza password, videosorveglianza
Commercio	una telecamera nella corsia della cassa riconosce gli articoli lasciati sotto il carrello
Mappe e telefoni	modelli 3D delle città negli *Earth viewer*; riconoscimento degli oggetti inquadrati con il telefono
Cinema	cattura della forma (*shape capture*) e del movimento (*motion capture*) degli attori per gli effetti speciali
Automobili	sistemi di visione per l'assistenza alla guida
Interazione e giochi	tracciamento a infrarossi nel controller del Wii, avatar 3D con il proprio volto
Spazio e robotica	rover su Marte (panorami, modelli 3D del terreno, rilevamento degli ostacoli, stima della posizione); RoboCup
Medicina	imaging 3D (risonanza magnetica, TAC), chirurgia guidata dalle immagini
Altro	veicoli autonomi, ausili per non vedenti, riconoscimento della lingua dei segni, controllo qualità nell'industria, ricerca di immagini

Tabella 1.5. Applicazioni della computer vision.

Le principali sedi di pubblicazione sono le riviste IEEE *Transactions on Pattern Analysis and Machine Intelligence* (TPAMI), *International Journal of Computer Vision* (IJCV) e IEEE *Transactions on Image Processing*. Tra le conferenze contano CVPR (*Conference on Computer Vision and Pattern Recognition*), che è annuale, e le biennali ICCV (*International Conference on Computer Vision*) ed ECCV (*European Conference on Computer Vision*). Le posizioni di queste riviste nelle classifiche mostrano il peso della computer vision nell'informatica e nell'ingegneria elettrica.

Capitolo 2

Acquisizione e immagine digitale

Un'immagine nasce da una catena di fenomeni. La luce di una sorgente colpisce gli oggetti della scena e ne viene in parte riflessa; una lente la proietta su un piano, dove un sensore la misura. Il risultato è una funzione continua che, per diventare un'immagine digitale, va campionata e quantizzata.

2.1 Dalla scena al sensore

Nell'acquisizione intervengono quattro elementi: una **sorgente di luce**, un **oggetto**, una **lente** e un **piano immagine**. La luce emessa dalla sorgente colpisce l'oggetto, che ne riflette una parte. La lente raccoglie la luce riflessa e la proietta sul piano immagine, dove si forma l'immagine (capovolta) dell'oggetto. In una fotocamera digitale il piano immagine è una matrice di sensori (*sensor array*), ognuno dei quali misura la luce che lo colpisce.

Quanta luce arriva al sensore dipende da due fattori: quanta ne illumina la scena e quanta l'oggetto ne riflette. La Tabella 2.1 dà qualche valore tipico dell'illuminazione prodotta da diverse sorgenti, misurata in *foot-candle* (un foot-candle vale circa 10,76 lux), e della riflettanza di diversi materiali. La riflettanza è la frazione della luce incidente che viene riflessa, e va da 0 (luce tutta assorbita) a 1 (luce tutta riflessa).

Sorgente	Illuminazione	Oggetto	Riflettanza
Luce solare	9000	Velluto nero	0,01
Cielo nuvoloso	1000	Parete bianca	0,80
Luce lunare	0,01	Argento e altri metalli	0,90
Luce d'ufficio	100	Neve	0,93

Tabella 2.1. Illuminazione di alcune sorgenti, in foot-candle, e riflettanza di alcuni materiali.

Quando la luce, di lunghezza d'onda λ, incontra una superficie, può comportarsi in cinque modi (Figura 2.1):

- **assorbimento** (*absorption*): l'energia viene assorbita dal materiale e non torna indietro;
- **diffusione** (*diffusion*): la luce viene riflessa in tutte le direzioni, come da una superficie opaca;

- **riflessione** speculare (*reflection*): la luce viene riflessa in una sola direzione, simmetrica a quella di arrivo, come da uno specchio;
- **trasparenza** (*transparency*): la luce attraversa il materiale senza cambiare direzione;
- **rifrazione** (*refraction*): la luce attraversa il materiale ma cambia direzione.

Una superficie reale combina questi comportamenti in proporzioni che dipendono da λ: per questo gli oggetti hanno colori diversi.

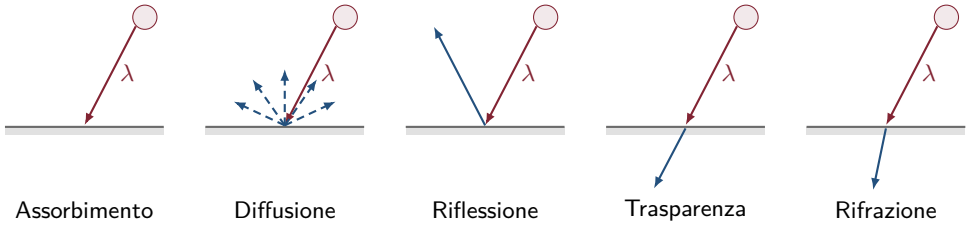


Figura 2.1. Che cosa può succedere alla luce di lunghezza d'onda λ quando incontra una superficie.

2.2 Il modello di formazione dell'immagine

Il modello matematico segue la luce passo per passo. Per le immagini naturali serve una sorgente di luce, con lunghezza d'onda λ.

- $E(x, y, z, \lambda)$ è la **luce incidente** nel punto della scena di coordinate (x, y, z), espresse nel sistema di riferimento del mondo.
- Ogni punto della scena ha una **funzione di riflettività** $r(x, y, z, \lambda)$.
- La luce riflessa dal punto, quella che il dispositivo di acquisizione cattura, è il prodotto delle due:

$$c(x, y, z, \lambda) = E(x, y, z, \lambda) \cdot r(x, y, z, \lambda). \tag{2.1}$$

È il prodotto tra illuminazione e riflettanza della sezione precedente, preso punto per punto e lunghezza d'onda per lunghezza d'onda. La camera trasforma poi c in un'immagine in due passi: una proiezione e una misura.

2.2.1 Proiezione

La **proiezione** $\mathcal{P}$ porta le coordinate del mondo (x, y, z) nelle coordinate della camera, o dell'immagine, (x', y'). La luce riflessa c diventa così la **luce proiettata** c_p, che dipende dalle coordinate dell'immagine:

$$c_p(x', y', \lambda) = \mathcal{P}\big(c(x, y, z, \lambda)\big). \tag{2.2}$$

Le proiezioni sono di due tipi.

- Nella **proiezione prospettica** (*perspective projection*) ciò che è più vicino appare più grande. È il modello dell'occhio umano e delle fotocamere: naturale, ma matematicamente più complicato.

- Nella **proiezione ortografica** (*orthographic projection*) la dimensione di un oggetto nell'immagine non dipende dalla sua distanza dal dispositivo. È innaturale, ma matematicamente più semplice.

La Figura 2.2 confronta le due proiezioni su due oggetti della stessa altezza, $\Delta_1 = \Delta_2$, posti a distanze $l_1 < l_2$.

- Nella proiezione prospettica tutti i raggi passano per un piccolo foro, il *pinhole* o foro stenopeico. L'oggetto più lontano è visto sotto un angolo più piccolo, quindi la sua immagine è più corta: $\delta_2 < \delta_1$.
- Nella proiezione ortografica i raggi arrivano paralleli all'asse e le due immagini sono uguali: $\delta_2 = \delta_1$.

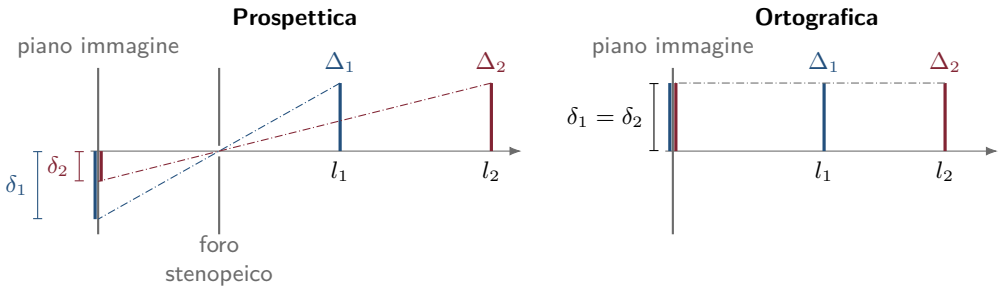


Figura 2.2. Due oggetti uguali a distanze diverse. Con la proiezione prospettica il più lontano ha un'immagine più piccola ($\delta_2 < \delta_1$), con quella ortografica le immagini sono uguali.

2.2.2 Sensibilità del sensore

Ottenuta $c_p(x', y', \lambda)$, entrano in gioco le caratteristiche del dispositivo di acquisizione. Ogni dispositivo ha una **funzione di sensibilità** $V(\lambda)$, che dice quanto il dispositivo è sensibile a ciascuna lunghezza d'onda presente in c_p. Può essere concentrata in una banda stretta intorno a una lunghezza d'onda λ_0 oppure, al limite, essere un impulso in λ_0: allora il sensore risponde solo a quella lunghezza d'onda. Il risultato è la **funzione immagine**, che dà la quantità di luce riflessa catturata nel punto (x', y') della camera:

$$f(x', y') = \int c_p(x', y', \lambda)\, V(\lambda)\, d\lambda. \tag{2.3}$$

L'integrale è una somma pesata: a ogni lunghezza d'onda, la luce che arriva nel punto si moltiplica per la sensibilità del sensore a quella lunghezza d'onda, e i prodotti si sommano. Con tre sole lunghezze d'onda, per esempio, se a 450, 550 e 650 nm arrivano 2, 4 e 6 unità di luce e la sensibilità vale 0, 1 e 0,5, il sensore misura $2 \cdot 0 + 4 \cdot 1 + 6 \cdot 0{,}5 = 7$: la luce a 450 nm non conta, quella a 650 nm conta a metà. L'integrale fa la stessa somma su tutte le lunghezze d'onda.

2.2.3 Immagini a colori

Manca ancora il colore. Una camera che cattura immagini a colori si può immaginare con tre sensori in ogni punto (x', y'), con funzioni di sensibilità V_R, V_G e V_B accordate

sul rosso, sul verde e sul blu. Ne escono tre funzioni immagine, una per canale:

$$f_C(x', y') = \int c_p(x', y', \lambda)\, V_C(\lambda)\, d\lambda, \qquad C \in \{R, G, B\}. \tag{2.4}$$

Sono queste tre funzioni che i dispositivi di visualizzazione, come un monitor, o l'occhio stesso usano per mostrare un'immagine «a colori». La Figura 2.3 mostra l'intera catena, compresa la digitalizzazione delle prossime sezioni.

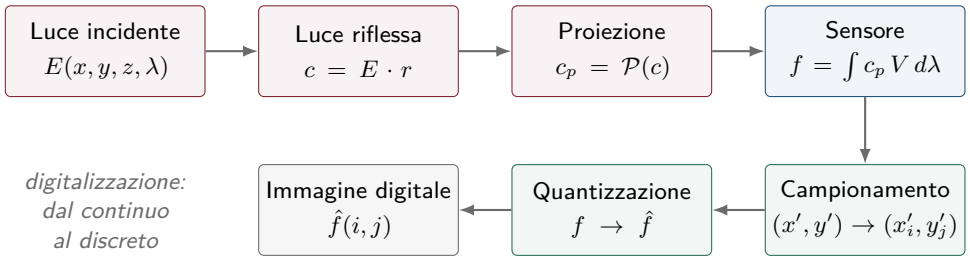


Figura 2.3. La catena che porta dalla luce incidente all'immagine digitale. Nella prima riga la formazione dell'immagine, ancora continua; nella seconda la digitalizzazione.

2.3 L'immagine digitale

L'**elaborazione digitale delle immagini** (*digital image processing*) tratta immagini multidimensionali con un calcolatore digitale. Per farlo, la funzione immagine va prima resa digitale.

> **Definizione (immagine digitale).** Un'**immagine digitale** è un array multidimensionale di valori. Ogni elemento dell'array è un **pixel** (*picture element*), e il suo valore è un numero per le immagini di intensità, un vettore per quelle a colori.

Un'immagine è quindi una funzione multidimensionale di coordinate spaziali: $f(x, y)$ nel caso 2D di una fotografia, $f(x, y, z)$ nel caso 3D di una TAC, $f(x, y, t)$ per un filmato, dove t è il tempo. Oltre all'intensità o al colore, il valore di f può essere anche un'altra grandezza. Per convenzione l'origine sta nell'angolo in alto a sinistra, con l'asse x verso destra e l'asse y verso il basso.

L'immagine proiettata, però, ha dominio e codominio continui:

$$(x', y') \in \mathbb{R}^2, \qquad f(x', y') \in \mathbb{R}.$$

Un calcolatore digitale non può elaborare parametri o funzioni che variano nel continuo, quindi bisogna **discretizzare** entrambi.

- Il **campionamento** (*sampling*) discretizza il dominio:

  $$(x', y') \to (x'_i, y'_j), \qquad i = 0, \dots, N-1, \quad j = 0, \dots, M-1,$$

  dove l'immagine ha M righe e N colonne, i numera le colonne e j le righe.

- La **quantizzazione** (*quantization*) discretizza i valori:

  $$f(x'_i, y'_j) \to \hat{f}(x'_i, y'_j) \in \mathbb{N}.$$

2.4 Campionamento

In una dimensione, il campionamento spaziale si può vedere come il prodotto del segnale continuo per una **funzione pettine** (*comb function*), cioè una sequenza di impulsi equispaziati. Il prodotto conserva il segnale solo nelle posizioni degli impulsi (Figura 2.4).

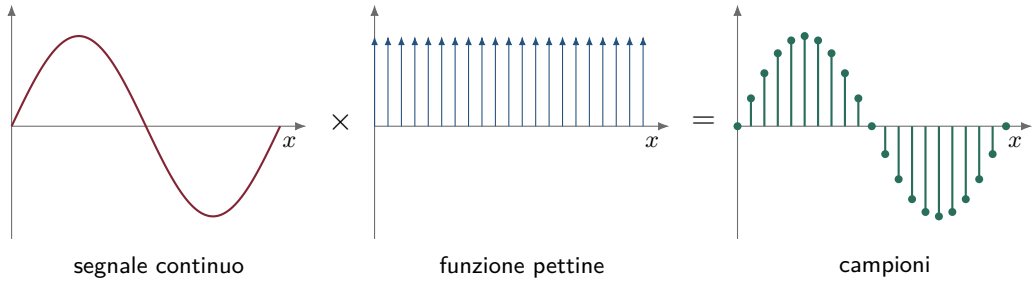


Figura 2.4. Il campionamento come prodotto del segnale continuo per la funzione pettine.

In due dimensioni il pettine diventa una griglia di impulsi, con passo Δ_x lungo x' e Δ_y lungo y':

$$\mathrm{comb}(x', y') = \sum_{i=0}^{N-1} \sum_{j=0}^{M-1} \delta(x' - i\Delta_x,\ y' - j\Delta_y), \tag{2.5}$$

dove δ è l'impulso di Dirac: un impulso concentrato nel punto in cui entrambi i suoi argomenti valgono zero, e nullo altrove. Il termine (i, j) della somma, quindi, è un impulso nel solo punto $(i\Delta_x, j\Delta_y)$. Con $\Delta_x = \Delta_y = 2$, per esempio, gli impulsi cadono in $(0, 0)$, $(2, 0)$, $(4, 0)$, $\ldots$, $(0, 2)$, $(2, 2)$, $\ldots$. L'immagine campionata è il prodotto

$$f(x'_i, y'_j) = f(x', y') \cdot \mathrm{comb}(x', y'), \tag{2.6}$$

che conserva f solo nei punti della griglia. Per semplicità si scrive $f(x'_i, y'_j) = f(i, j)$, indicizzando i campioni con gli interi (i, j).

Fisicamente, ogni sensore della matrice memorizza il colore medio della luce che lo colpisce. Il pixel, quindi, non corrisponde a un punto ma a un'area: è la più piccola porzione di immagine che si possa trattare.

2.5 Quantizzazione

La **quantizzazione** converte i valori continui dei pixel in numeri discreti. Il numero di livelli disponibili, di grigio o di colore, si chiama **risoluzione cromatica** o **profondità di colore** (*color resolution*, *color depth*) e si esprime anche con il numero di bit usati per ogni pixel. Con k bit per pixel i livelli sono

$$L = 2^k. \tag{2.7}$$

La **funzione di quantizzazione** è una funzione a gradini. Divide l'intervallo delle intensità luminose, dalla più scura alla più chiara, in L intervalli e assegna a

ciascuno un livello tra 0 e $L-1$ (Figura 2.5a). Applicata a un segnale campionato, sostituisce ogni campione $f(i)$ con il livello del suo intervallo, $\hat{f}(i)$ (Figura 2.5b). Con $L=4$ e intensità tra 0 e 1, per esempio, gli intervalli sono [0; 0,25), [0,25; 0,5), [0,5; 0,75) e [0,75; 1], e un campione che vale 0,6 diventa il livello 2.

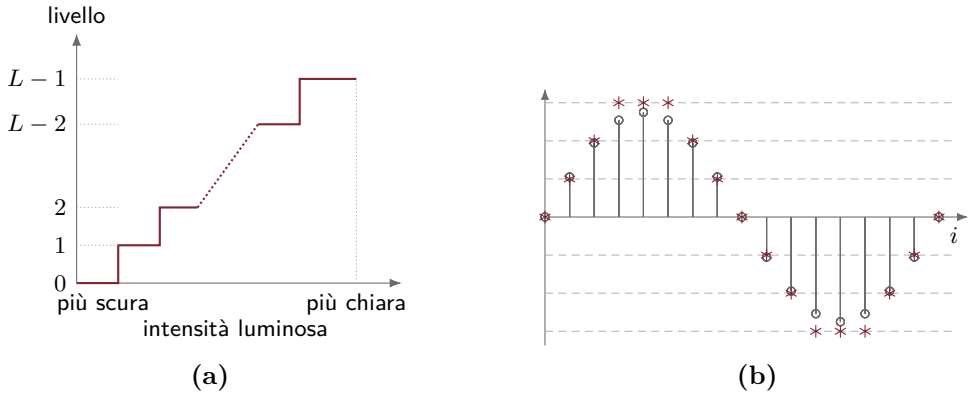


Figura 2.5. (a) La funzione di quantizzazione. (b) I campioni $f(i)$ di un segnale (cerchi grigi) e i valori quantizzati $\hat{f}(i)$ (asterischi rossi); le linee tratteggiate sono i livelli.

Diminuendo i livelli l'immagine peggiora. Scendendo da 256 a 32 livelli le differenze sono minime; sotto, la qualità cala. Con 4 livelli compaiono con evidenza i **falsi contorni** (*false contours*): nelle zone in cui l'intensità varia lentamente, il salto da un livello all'altro disegna bordi che nella scena non esistono. Con 2 livelli resta un'immagine binaria.

Dopo campionamento e quantizzazione l'immagine è una funzione discreta sia nel dominio sia nei valori:

$$\hat{f} : \Omega \to \{0, 1, \ldots, L-1\},$$

dove $\Omega \subset \mathbb{N}^2$ è l'insieme delle posizioni (i, j) dei pixel. Tipicamente $L = 2^8 = 256$, cioè $k = 8$: una quantizzazione a 8 bit, con i livelli da 0 a 255.

Dimensione e profondità adatte. Quale combinazione di numero di pixel e di livelli sia «adatta» è soggettivo: dipende dal soggetto dell'immagine e da chi la guarda. Confrontando tre immagini con poco, medio e molto dettaglio (un volto in primo piano, un fotografo con il treppiede, una folla) se ne ricavano due indicazioni per soddisfare un osservatore umano:

1. a parità di dimensione, un'immagine con poco dettaglio può richiedere più livelli, perché nelle sue ampie zone uniformi i falsi contorni si vedono subito;
2. al crescere della dimensione dell'immagine possono bastare meno livelli di grigio.

2.6 Rappresentazione e tipi di immagine

La struttura dati di un'immagine è semplicemente un array 2D di valori, di un tipo qualsiasi: bit, byte, interi, numeri in virgola mobile. Il pixel $\hat{f}(i, j)$ sta nella colonna i e nella riga j (Figura 2.6).

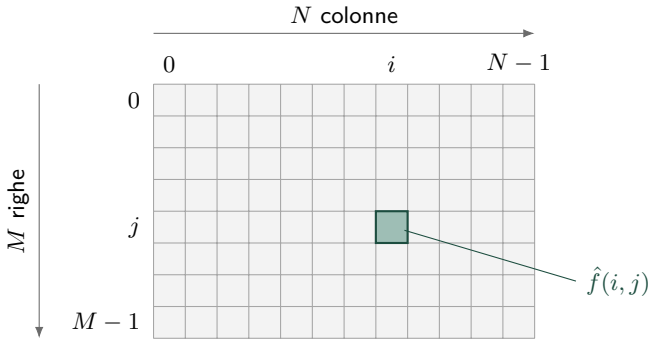


Figura 2.6. Un'immagine come array di M righe e N colonne.

Secondo il contenuto dei pixel si distinguono i quattro tipi di immagine della Tabella 2.2.

Tipo	Contenuto di un pixel
Di intensità (monocromatica)	un numero, l'intensità della luce, di solito rappresentato come livello di grigio
RGB (a colori)	un vettore con le componenti rossa, verde e blu; l'immagine equivale a tre matrici, una per canale
Binaria (in bianco e nero)	un bit: 1 per il bianco, 0 per il nero
Indicizzata	un indice che punta a un colore in una tabella dei colori (*color table*); ogni riga della tabella dà le componenti R, G, B (per esempio, rosso puro per $R = 1$, $G = B = 0$). Si usa, fra l'altro, per le immagini mediche in falsi colori

Tabella 2.2. Tipi di immagine digitale.

Memoria. Un'immagine di M righe e N colonne ha $M \cdot N$ pixel. Con L livelli di intensità ogni pixel richiede $k = \log_2 L$ bit, cioè l'esponente per cui $2^k = L$ (Equazione (2.7)), e l'intera immagine

$$b = M \cdot N \cdot k \text{ bit}; \tag{2.8}$$

se l'immagine è quadrata ($M = N$), $b = N^2 k$. Un'immagine di 1024×1024 pixel a 256 livelli ($k = 8$), per esempio, occupa $1024^2 \cdot 8 = 8.388.608$ bit, cioè 1.048.576 byte.

2.7 Risoluzione spaziale

Il campionamento discretizza l'immagine nel dominio spaziale. Da esso dipende la risoluzione spaziale, o risoluzione dell'immagine, legata alla dimensione dei pixel e quindi al loro numero.

> **Definizione (risoluzione spaziale).** La **risoluzione spaziale** è il più piccolo dettaglio distinguibile in un'immagine. Si esprime in pixel per pollice (ppi, *pixel per inch*) o in punti per pollice (dpi, *dot per inch*).

Le sole dimensioni $M \times N$ non bastano a esprimerla. Non dicono quale porzione della scena i pixel coprono, e quindi quale livello di dettaglio della scena originale riescono a rappresentare.

Sottocampionamento. Se i punti di campionamento sono troppo radi rispetto ai dettagli, questi si perdono. Si pensi a una scena fatta di strisce verticali larghe 1 mm, alternate chiare e scure, campionata con un campione ogni 2 mm: i campioni cadono tutti su strisce dello stesso colore. Se cadono sulle strisce scure, l'immagine campionata è uniformemente scura; se cadono sulle chiare, è uniformemente chiara. In entrambi i casi le strisce sono sparite.

Effetti della risoluzione. Riducendo il numero di pixel, per esempio da 256×256 a 32×32, i dettagli si perdono e l'immagine diventa a blocchi. Se la dimensione dei pixel resta la stessa, cambiare risoluzione cambia la dimensione dell'immagine: a 512, 256, 128… pixel di lato l'immagine si rimpicciolisce. Se invece la si mostra sempre alla stessa dimensione, replicando righe e colonne, i pixel diventano grandi e visibili:

- ad alta risoluzione l'immagine appare continua, finché i pixel sono più piccoli della risoluzione spaziale del sistema visivo umano;
- a bassa risoluzione si vedono i contorni dei singoli pixel.

Il dettaglio perso non si recupera ingrandendo di nuovo l'immagine, perché il **sottocampionamento** (*down sampling*) è un processo irreversibile: un'interpolazione più accurata rende il risultato più morbido, ma i dettagli non tornano.

Quale risoluzione. La risoluzione migliore dipende dall'applicazione: per la visualizzazione a schermo e sul web bastano 72 dpi, la risoluzione di un monitor, mentre per la stampa si arriva a 1200 dpi. Per passare da una risoluzione bassa a una alta si interpola, sapendo che il risultato è solo un'approssimazione. Lo **zoom** di un'immagine ha due fasi:

1. la creazione della nuova griglia: ingrandendo di 1,5 volte un'immagine di 500×500 pixel se ne ottiene una di 750×750;
2. l'assegnazione di un livello di grigio a ogni nuovo pixel, cioè l'interpolazione.

2.8 Relazioni tra pixel

In questa sezione (x, y) sono le coordinate intere di un pixel: x è la colonna e y la riga, con l'origine $(0, 0)$ in alto a sinistra. Un pixel p in (x, y) è circondato da otto pixel, di coordinate $(x \pm 1, y)$, $(x, y \pm 1)$ e $(x \pm 1, y \pm 1)$.

2.8.1 Vicinati

La relazione di vicinato dice quali pixel sono adiacenti e serve ad analizzare le regioni. Se ne usano tre (Figura 2.7):

- i **4-vicini** di p, solo in orizzontale e in verticale:

$$N_4(p) = \{(x-1, y),\ (x+1, y),\ (x, y-1),\ (x, y+1)\};$$

- i **vicini diagonali**:

$$N_D(p) = \{(x-1, y-1),\ (x+1, y-1),\ (x-1, y+1),\ (x+1, y+1)\};$$

- gli **8-vicini**, cioè tutti i pixel attorno: $N_8(p) = N_4(p) \cup N_D(p)$.

La relazione è simmetrica: $q \in N_4(p)$ implica $p \in N_4(q)$, e lo stesso vale per gli altri vicinati.

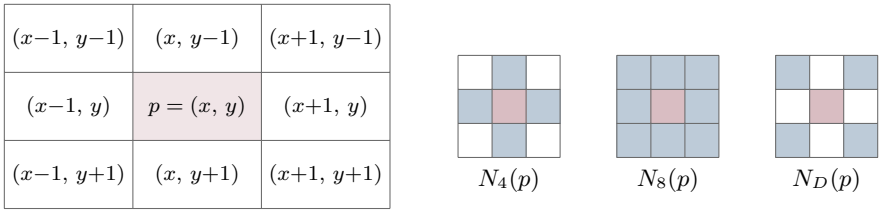


Figura 2.7. I 4-vicini, gli 8-vicini e i vicini diagonali del pixel p.

2.8.2 Connettività, adiacenza e percorsi

La connettività deriva dalla relazione di vicinato. Due pixel sono **connessi** se appartengono alla stessa classe (per esempio hanno lo stesso colore, o un'intensità nello stesso intervallo) e sono vicini l'uno dell'altro. Per p e q della stessa classe:

- p e q sono **4-connessi** se $q \in N_4(p)$;
- sono **8-connessi** se $q \in N_8(p)$;
- sono **m-connessi** (connettività mista) se $q \in N_4(p)$, oppure se $q \in N_D(p)$ e nessun pixel di $N_4(p) \cap N_4(q)$ appartiene alla stessa classe.

Perché la condizione guarda la classe. Due vicini diagonali p e q hanno sempre due 4-vicini in comune: i due pixel che completano il quadrato 2×2 di cui p e q sono angoli opposti. L'insieme $N_4(p) \cap N_4(q)$, quindi, non è mai vuoto, e la condizione riguarda la classe dei suoi due pixel. Se uno dei due è della stessa classe, p e q sono già collegati passando per quel pixel, e il collegamento diagonale sarebbe un doppione. Così la m-connettività elimina i collegamenti

doppi.

Un pixel p è **adiacente** a un pixel q se sono connessi. Due sottoinsiemi S_1 e S_2 dell'immagine sono adiacenti se qualche pixel di S_1 è adiacente a qualche pixel di S_2. Secondo il tipo di connettività si parla di 4-adiacenza, 8-adiacenza o m-adiacenza.

Definizione (percorso). Un **percorso** (*path*) dal pixel p in (x, y) al pixel q in (s, t) è una sequenza di pixel distinti

$$(x_0, y_0),\ (x_1, y_1),\ \ldots,\ (x_n, y_n)$$

tale che $(x_0, y_0) = (x, y)$, $(x_n, y_n) = (s, t)$ e (x_i, y_i) è adiacente a (x_{i-1}, y_{i-1}) per $i = 1, \ldots, n$.

Anche i percorsi sono 4-, 8- o m-percorsi, secondo l'adiacenza usata. Gli 8-percorsi possono essere ambigui (Figura 2.8): tra p e q ci sono due strade, una che passa per il 4-vicino comune e una che taglia in diagonale. L'm-adiacenza toglie il collegamento diagonale proprio in questi casi, e il percorso diventa unico.

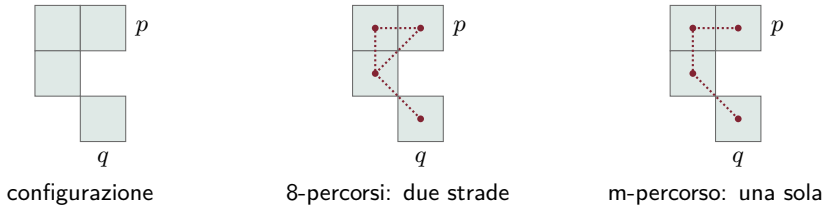


Figura 2.8. Percorsi da p a q. Con l'8-adiacenza il primo passo può tagliare in diagonale oppure passare per il pixel a sinistra di p. Con l'm-adiacenza quel taglio non è ammesso, perché il pixel a sinistra di p è un 4-vicino comune della stessa classe, e il percorso è unico.

2.8.3 Distanze

Definizione (distanza). Una funzione D è una **distanza** (o **metrica**) se, per tre pixel qualsiasi p, q, z:

- $D(p, q) \geq 0$, e $D(p, q) = 0$ se e solo se $p = q$: la distanza non è mai negativa, ed è zero solo da un pixel a sé stesso;
- $D(p, q) = D(q, p)$: da p a q c'è quanto da q a p;
- $D(p, z) \leq D(p, q) + D(q, z)$: passare per un terzo pixel non accorcia la strada (disuguaglianza triangolare).

Per p in (x, y) e q in (s, t), la più familiare è la **distanza euclidea**:

$$D_e(p, q) = \sqrt{(x - s)^2 + (y - t)^2}. \tag{2.9}$$

Sulla griglia dei pixel se ne usano altre due (Figura 2.9).

- La **distanza** D_4, o *city-block distance*, conta i passi orizzontali e verticali, come chi si muove tra gli isolati di una città:

$$D_4(p,q) = |x-s| + |y-t|.$$

  I pixel a distanza $D_4 \le r$ da p formano un rombo, e quelli con $D_4 = 1$ sono i 4-vicini di p.

- La **distanza** D_8, o *chessboard distance*, conta anche i passi diagonali, come le mosse del re sulla scacchiera:

$$D_8(p,q) = \max\big(|x-s|,\ |y-t|\big).$$

  I pixel a distanza $D_8 \le r$ da p formano un quadrato, e quelli con $D_8 = 1$ sono gli 8-vicini di p.

Per $p = (0,0)$ e $q = (3,1)$, per esempio, le tre distanze sono diverse:

- $D_e = \sqrt{3^2+1^2} = \sqrt{10} \approx 3{,}16$, la linea retta;
- $D_4 = 3+1 = 4$, perché servono tre passi orizzontali e uno verticale;
- $D_8 = \max(3,1) = 3$, perché il passo verticale si fa insieme a uno orizzontale, in diagonale.

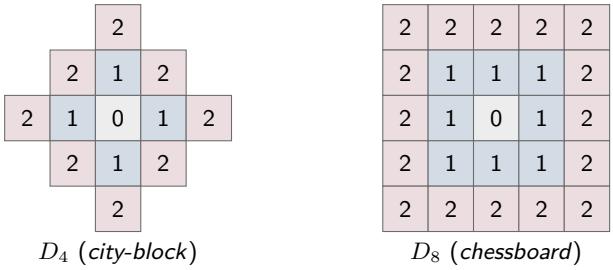


Figura 2.9. Distanze D_4 e D_8 dal pixel centrale, fino a 2.

2.9 Ridimensionamento e interpolazione

Lo zoom della Sezione 2.7 ha due fasi: la creazione della nuova griglia e l'assegnazione di un valore a ogni nuovo pixel. L'algoritmo di **ridimensionamento** le realizza scorrendo i pixel della *nuova* immagine. Per ciascuno:

1. ne mappa le coordinate nel sistema della vecchia immagine;
2. stima il valore della vecchia immagine in quel punto;
3. assegna il valore al pixel della nuova immagine.

Si parte dai pixel nuovi perché così ognuno riceve un valore. Partendo dai vecchi, un ingrandimento lascerebbe dei buchi: con un fattore 1,5, le colonne vecchie 0, 1 e 2 finirebbero nelle colonne nuove 0; 1,5 e 3, e una tra le colonne nuove 1 e 2 resterebbe vuota.

Il passo delicato è il secondo. Nel seguito $\mathrm{Im}(x,y,z)$ è il valore della vecchia immagine nella colonna x, nella riga y e nel canale z, ed è definito solo per coordinate

intere, come $(25, 45, 3)$. Ingrandendo di 1,5 volte un'immagine di 500×500 pixel, il pixel nuovo $(300, 450)$ corrisponde al punto $(200, 300)$ della vecchia immagine, che è un pixel. Il pixel nuovo $(301, 450)$ corrisponde invece a (200,67; 300), che cade tra due pixel: lì il valore va stimato **interpolando** i pixel vicini. Il canale resta sempre intero: si interpola solo nello spazio.

Nearest neighbor. Il metodo più semplice prende il valore del pixel più vicino (*nearest neighbor*), arrotondando le coordinate all'intero più vicino:

$$f(x, y, z) = \mathrm{Im}(\mathrm{round}(x),\ \mathrm{round}(y),\ z), \tag{2.10}$$

dove $f(x, y, z)$ è il valore stimato. Il punto (200,67; 300), per esempio, prende il valore del pixel $(201, 300)$. Il risultato è a blocchi. Un errore frequente, in C, è calcolare la coordinata con una divisione tra interi, che tronca invece di arrotondare: $301 \cdot 500/750$ dà 200, non 201.

Interpolazione su triangoli. A volte i valori noti non stanno su una griglia regolare. Allora si cercano triangoli: si trova il triangolo di punti noti V_1, V_2, V_3 che contiene il punto q da stimare e si fa una media pesata dei tre valori. Nelle formule V_1, V_2, V_3 indicano anche i valori noti nei vertici, e q il valore stimato. Il peso di ogni vertice è l'area A_1, A_2 o A_3 del sottotriangolo opposto, formato da q e dagli altri due vertici (Figura 2.10a):

$$q = \frac{A_1 V_1 + A_2 V_2 + A_3 V_3}{A_1 + A_2 + A_3}. \tag{2.11}$$

Il vertice più lontano da q ha l'area opposta più piccola, quindi il peso minore, e il più vicino ha il peso maggiore. Il denominatore, l'area totale, serve perché il risultato sia davvero una media dei tre valori. Con aree $A_1 = 1$, $A_2 = 2$, $A_3 = 3$ e valori $V_1 = 60$, $V_2 = 30$, $V_3 = 0$, per esempio, la sola somma pesata dà $60 + 60 + 0 = 120$, più di qualunque valore dei vertici; divisa per l'area totale, 6, dà 20, che sta tra 0 e 60 come deve fare una media.

Interpolazione bilineare. Su una griglia regolare c'è un metodo più semplice, che usa i quattro pixel più vicini, cioè i vertici del quadrato che contiene q: V_1 in alto a sinistra, V_2 in alto a destra, V_3 in basso a sinistra e V_4 in basso a destra. L'idea è la stessa, una somma pesata in cui il peso di ogni vertice è l'area del rettangolo opposto (Figura 2.10b):

$$q = V_1 A_1 + V_2 A_2 + V_3 A_3 + V_4 A_4, \tag{2.12}$$

dove i pesi sono le aree

$$A_1 = d_2 d_4, \qquad A_2 = d_1 d_4, \qquad A_3 = d_2 d_3, \qquad A_4 = d_1 d_3.$$

Qui d_1 e d_2 sono le distanze orizzontali di q dalla colonna di sinistra e da quella di destra, d_3 e d_4 quelle verticali dalla riga in alto e da quella in basso. Poiché i pixel distano 1, $d_1 + d_2 = 1$ e $d_3 + d_4 = 1$: le quattro aree sommano a 1 e non serve normalizzare.

Con i numeri: siano $V_1 = 10$, $V_2 = 20$, $V_3 = 30$, $V_4 = 40$, e sia q a 0,3 dalla colonna di sinistra e a 0,6 dalla riga in alto. Allora $d_1 = 0{,}3$, $d_2 = 0{,}7$, $d_3 = 0{,}6$, $d_4 = 0{,}4$, e

Vertice	Posizione	Peso	Contributo
$V_1 = 10$	in alto a sinistra	$A_1 = 0{,}7 \cdot 0{,}4 = 0{,}28$	2,8
$V_2 = 20$	in alto a destra	$A_2 = 0{,}3 \cdot 0{,}4 = 0{,}12$	2,4
$V_3 = 30$	in basso a sinistra	$A_3 = 0{,}7 \cdot 0{,}6 = 0{,}42$	12,6
$V_4 = 40$	in basso a destra	$A_4 = 0{,}3 \cdot 0{,}6 = 0{,}18$	7,2
		totale 1	$q = 25$

Il peso maggiore va a V_3, il vertice più vicino a q, che sta più vicino alla colonna di sinistra e alla riga in basso; il minore a V_2, il più lontano.

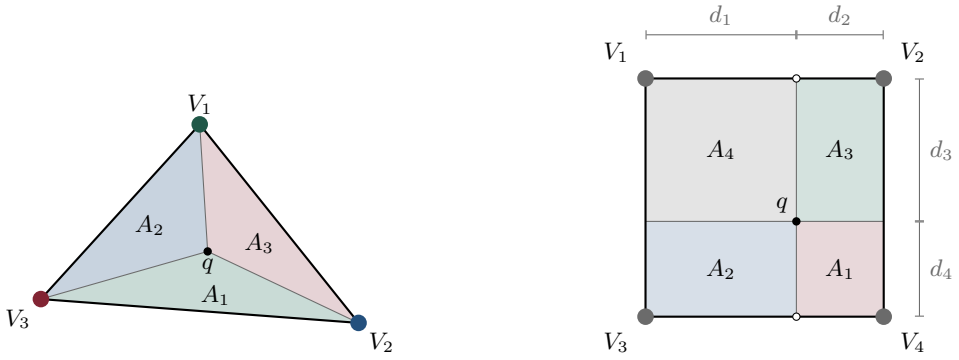


Figura 2.10. (a) Interpolazione su triangoli: ogni vertice pesa quanto l'area del sottotriangolo opposto. (b) Interpolazione bilineare: ogni vertice pesa quanto l'area del rettangolo opposto.

Lo stesso risultato si ottiene con due interpolazioni lineari successive. Prima si interpola lungo x sul lato alto e su quello basso del quadrato, e si ottengono i valori nei punti q_1 e q_2, sopra e sotto q:

$$q_1 = d_2 V_1 + d_1 V_2, \qquad q_2 = d_2 V_3 + d_1 V_4.$$

Ogni valore pesa quanto la distanza di q dal lato opposto: V_1, a sinistra, pesa d_2, la distanza da destra, quindi conta di più quanto più q gli è vicino. Poi si interpola lungo y tra i due, con la stessa regola: $q = d_4\, q_1 + d_3\, q_2$. Con i numeri dell'esempio, $q_1 = 0{,}7 \cdot 10 + 0{,}3 \cdot 20 = 13$, $q_2 = 0{,}7 \cdot 30 + 0{,}3 \cdot 40 = 33$ e $q = 0{,}4 \cdot 13 + 0{,}6 \cdot 33 = 25$, come prima. Non è un caso: moltiplicando, i pesi dei quattro vertici sono proprio le aree dell'Equazione (2.12),

$$q = d_4(d_2 V_1 + d_1 V_2) + d_3(d_2 V_3 + d_1 V_4) = d_2 d_4\, V_1 + d_1 d_4\, V_2 + d_2 d_3\, V_3 + d_1 d_3\, V_4.$$

L'interpolazione bilineare è più morbida del nearest neighbor e un po' più costosa: quattro letture in memoria e qualche operazione. Spesso è il giusto compromesso tra velocità e qualità.

Interpolazione bicubica. È più complessa, e forse migliore. Usa i 16 pixel più vicini, una finestra 4×4, e polinomi di terzo grado,

$$f(x) = a + bx + cx^2 + dx^3.$$

Si interpola lungo un asse, adattando un polinomio cubico a ciascuna delle quattro righe della finestra; poi si adatta un altro polinomio ai quattro valori ottenuti. È dunque un'interpolazione cubica di quattro interpolazioni cubiche. Il risultato è più morbido della bilineare e non ha i suoi artefatti «a stella». La Tabella 2.3 confronta i metodi.

Metodo	Pixel usati	Caratteristiche
Nearest neighbor	1	il più veloce, ma a blocchi
Bilineare	4 (2×2)	più morbida; spesso il giusto compromesso
Bicubica	16 (4×4)	ancora più morbida, senza artefatti a stella; la più costosa
Su triangoli	3	per valori noti che non stanno su una griglia

Tabella 2.3. Metodi di interpolazione a confronto.

Capitolo 3

Il colore

Nel Capitolo 2 la luce partiva da una sorgente, veniva riflessa da un punto di una superficie e la lente la portava sul piano immagine. Il colore nasce lungo questa stessa catena.

3.1 Che cos'è il colore

Definizione (colore). Il **colore** è una proprietà psicologica della nostra esperienza visiva quando guardiamo oggetti e luci, non una proprietà fisica di quegli oggetti o di quelle luci. È il risultato dell'interazione tra la luce fisica presente nell'ambiente e il nostro sistema visivo.

La luce visibile è una piccola parte dello spettro elettromagnetico. Le lunghezze d'onda λ vanno da circa 400 nm (violetto) a circa 700 nm (rosso). Più in basso ci sono ultravioletto, raggi X e raggi gamma, più in alto infrarosso, microonde e onde radio. Anche dentro questo intervallo l'occhio non è ugualmente sensibile a tutte le lunghezze d'onda. La sua **funzione di sensibilità alla luminanza** $V(\lambda)$ (*luminance sensitivity function*), l'analoga per l'occhio della sensibilità di un sensore, è massima intorno ai 555 nm, nel verde-giallo, e cala verso i due estremi (Figura 3.1). Vediamo proprio queste lunghezze d'onda perché è lì che il Sole irradia la sua energia elettromagnetica.

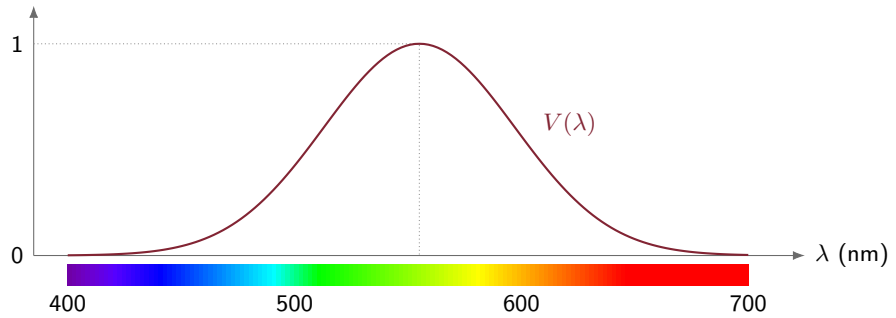


Figura 3.1. Lo spettro visibile e la sensibilità dell'occhio alla luminanza $V(\lambda)$ (curva indicativa).

3.2 Luce e superfici

Una sorgente di luce è descritta completamente, dal punto di vista fisico, dal suo **spettro**: la quantità di energia emessa nell'unità di tempo a ogni lunghezza d'onda tra 400 e 700 nm. Lo si rappresenta come **potenza spettrale relativa** (*relative spectral power*) in funzione di λ. La Tabella 3.1 descrive gli spettri di quattro sorgenti.

Sorgente	Spettro
Laser a rubino	una riga strettissima nel rosso, vicino ai 700 nm
Cristallo di fosfuro di gallio	una banda stretta intorno ai 560 nm, nel giallo-verde
Lampadina a tungsteno	potenza che cresce con la lunghezza d'onda: molto rosso, poco blu
Luce del giorno	più o meno piatto su tutto il visibile

Tabella 3.1. Spettri di alcune sorgenti di luce.

Le superfici si descrivono con lo **spettro di riflettanza**, la frazione di luce riflessa a ogni lunghezza d'onda. Quattro esempi:

- Un pomodoro (rosso) riflette quasi solo sopra i 600 nm.
- Una banana (gialla) riflette da circa 500 nm in su, cioè verde e rosso insieme.
- I mirtilli (blu) riflettono soprattutto tra 400 e 500 nm.
- L'uva (viola) riflette ai due estremi, nel blu e nel rosso.

Il colore osservato nasce dall'interazione tra lo spettro della sorgente e la riflettanza della superficie. Lunghezza d'onda per lunghezza d'onda, il **segnale di colore** che arriva all'occhio è il prodotto dell'illuminazione per la riflettanza (Figura 3.2). È il prodotto $c = E \cdot r$ dell'Equazione (2.1), con E la luce incidente, r la riflettività della superficie e c la luce riflessa, che qui è il segnale di colore. Si parla allora di **radiometria spettrale**: tutte le definizioni e le unità di misura si intendono «per unità di lunghezza d'onda», e tutte le grandezze diventano spettrali.

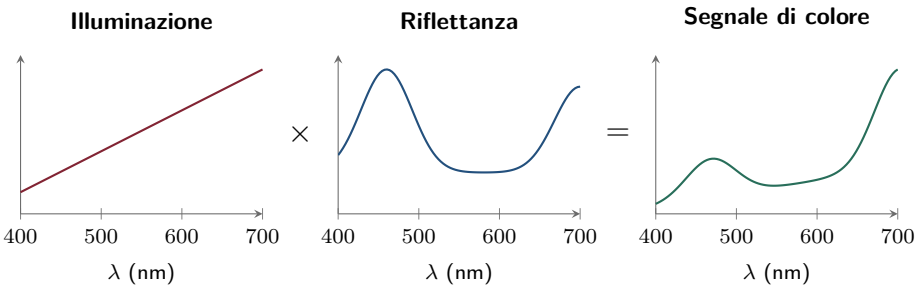


Figura 3.2. Il segnale di colore è il prodotto, lunghezza d'onda per lunghezza d'onda, dello spettro dell'illuminazione e della riflettanza della superficie (curve indicative).

3.3 L'occhio

L'occhio umano funziona come una fotocamera.

- L'**iride** è l'anello colorato, con muscoli radiali.
- La **pupilla** è il foro, cioè l'apertura, e la sua dimensione è controllata dall'iride.
- Il **cristallino** (*lens*) è la lente: cambia forma grazie ai muscoli ciliari, per mettere a fuoco oggetti a distanze diverse.
- La «pellicola» è la **retina**, con le cellule fotorecettrici: i **bastoncelli** (*rods*) e i **coni** (*cones*).

I fotorecettori non sono distribuiti in modo uniforme sulla retina. I bastoncelli rispondono all'intensità della luce, i coni al colore. La **fovea** è una piccola regione, di 1 o 2 gradi, al centro del campo visivo: ha la massima densità di coni e nessun bastoncello. In periferia l'acuità visiva è minore, perché molti bastoncelli sono collegati allo stesso neurone. Nel **punto cieco** (*blind spot*), dove il nervo ottico lascia l'occhio, non ci sono fotorecettori.

Le due famiglie lavorano in condizioni di luce diverse. L'intensità della luce riflessa dagli oggetti varia di molti ordini di grandezza, dalla soglia percepibile al buio, a occhio adattato, fino alla luce abbagliante del sole sulla neve. Sotto la soglia della percezione del colore, che corrisponde più o meno alla luce di una notte di luna piena, lavorano solo i bastoncelli e la visione è senza colori. Sopra entrano in gioco i coni. Ecco perché al buio non si riesce a leggere: lavorano solo i bastoncelli, che mancano nella fovea e, collegati a gruppi allo stesso neurone, danno poca acuità.

I coni sono di tre tipi, ciascuno sensibile a una diversa banda dello spettro (Figura 3.3):

- i coni **S** (*short*), con il picco a 437 nm, nel blu;
- i coni **M** (*medium*), con il picco a 533 nm, nel verde;
- i coni **L** (*long*), con il picco a 564 nm, verso il rosso.

I bastoncelli hanno il picco a 498 nm. I coni L, M e S stanno in un rapporto di circa 10:5:1, e al centro della fovea i coni S quasi mancano. Sulla retina i tre tipi formano un mosaico irregolare. Ogni punto della retina, quindi, misura un solo tipo di risposta, e il sistema visivo deve ricostruire il colore interpolando campioni spazialmente sfalsati. Ne sono una prova i **colori di Brewster**: colori illusori che si vedono guardando fitti motivi in bianco e nero.

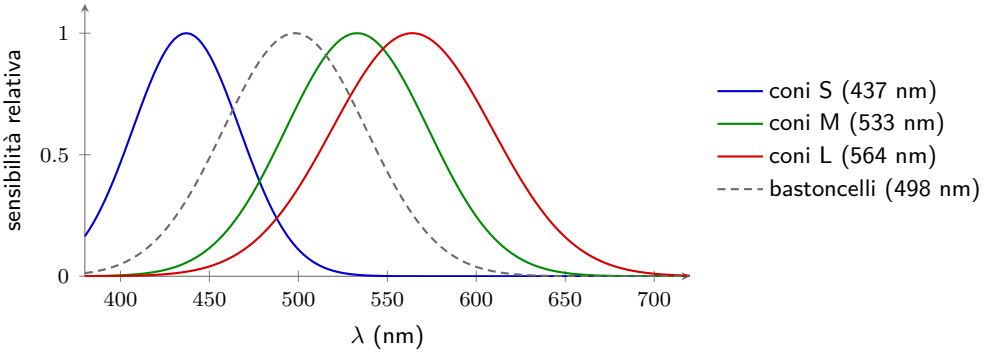


Figura 3.3. Sensibilità relative dei tre tipi di coni e dei bastoncelli (curve indicative).

3.4 Percezione del colore e metameri

Coni e bastoncelli agiscono come filtri sullo spettro. Per ottenere l'uscita di un filtro si moltiplica la sua curva di risposta per lo spettro e si integra su tutte le lunghezze d'onda. Se $t(\lambda)$ è lo spettro che arriva all'occhio e $\sigma_K(\lambda)$ la sensibilità dei coni di tipo $K \in \{S, M, L\}$, la risposta di quei coni è

$$\rho_K = \int t(\lambda)\, \sigma_K(\lambda)\, d\lambda, \tag{3.1}$$

la stessa forma delle funzioni immagine f_C dell'Equazione (2.4), le risposte dei sensori R, G e B di una camera. Anche qui l'integrale è una somma pesata: a ogni lunghezza d'onda la luce si moltiplica per la sensibilità del cono, e i prodotti si sommano. Ogni tipo di cono, quindi, restituisce un solo numero.

Come si può rappresentare un intero spettro con tre numeri? Non si può: la maggior parte dell'informazione va persa. Di conseguenza, due spettri diversi possono apparire indistinguibili.

Definizione (metameri). In colorimetria, il **metamerismo** è la corrispondenza percepita tra colori con distribuzioni spettrali di potenza diverse. I colori che si corrispondono in questo modo si chiamano **metameri**.

Esempio (giallo puro e giallo composto). Una luce monocromatica gialla ha lo spettro concentrato in una sola riga. Una miscela di luce verde e rossa ha uno spettro con due picchi, nel verde e nel rosso. Moltiplicati per le sensibilità dei coni e integrati, i due spettri possono dare le stesse tre risposte ρ_S, ρ_M, ρ_L: le due luci appaiono dello stesso giallo. Lo stesso succede con le superfici: un petalo bianco e un fiore bianco possono avere spettri di riflettanza molto diversi, eppure apparire dello stesso colore.

3.5 Colorimetria

Si vuole capire quali spettri producono la stessa sensazione di colore in persone diverse, in condizioni di osservazione simili. Lo strumento sono gli **esperimenti di color matching** (corrispondenza di colori).

- L'osservatore guarda uno schermo bianco diviso in due metà.
- Su una metà cade la luce di prova (*test light*). Sull'altra cade la sovrapposizione di tre luci primarie, di cui l'osservatore regola l'intensità.
- L'osservatore regola le tre intensità finché le due metà non appaiono identiche.

Gli esperimenti mostrano che tre numeri bastano a codificare il colore, cioè la **tricromia**. La maggior parte delle persone riesce a riprodurre qualunque luce con tre primarie (l'eccezione è il daltonismo), e, a parità di luce e di primarie, quasi tutti scelgono gli stessi pesi.

Leggi di Grassmann. La corrispondenza additiva è lineare. Nelle formule che seguono P_1, P_2, P_3 sono le primarie e il segno $=$ indica che le due luci appaiono identiche.

- Se due luci di prova sono riprodotte con gli stessi pesi, sono uguali tra loro: se $A = u_1P_1 + u_2P_2 + u_3P_3$ e $B = u_1P_1 + u_2P_2 + u_3P_3$, allora $A = B$.
- Se si mescolano due luci, i pesi si sommano: se $A = u_1P_1 + u_2P_2 + u_3P_3$ e $B = v_1P_1 + v_2P_2 + v_3P_3$, allora

$$A + B = (u_1 + v_1)P_1 + (u_2 + v_2)P_2 + (u_3 + v_3)P_3.$$

- Se si scala la luce, i pesi si scalano dello stesso fattore: se $A = u_1P_1+u_2P_2+u_3P_3$, allora $kA = (ku_1)P_1 + (ku_2)P_2 + (ku_3)P_3$.

Con i numeri: se $A = 2P_1 + 1P_2 + 0P_3$ e $B = 1P_1 + 3P_2 + 1P_3$, la miscela delle due luci si riproduce con $A + B = 3P_1 + 4P_2 + 1P_3$, e la luce A con intensità doppia con $2A = 4P_1 + 2P_2 + 0P_3$. Le leggi valgono perché le risposte dei coni sono lineari: ogni risposta è una somma pesata dello spettro (Equazione (3.1)), quindi a due luci sovrapposte risponde la somma delle due risposte, e a una luce doppia il doppio della risposta.

Spazi colore lineari. Uno **spazio colore lineare** si definisce scegliendo tre primarie. Le coordinate di un colore sono i pesi delle primarie che lo riproducono. Mescolare luci, allora, corrisponde a fare combinazioni lineari (Figura 3.4):

- mescolando due luci si ottengono i colori del segmento che le unisce;
- mescolando tre luci si ottengono i colori del triangolo che definiscono.

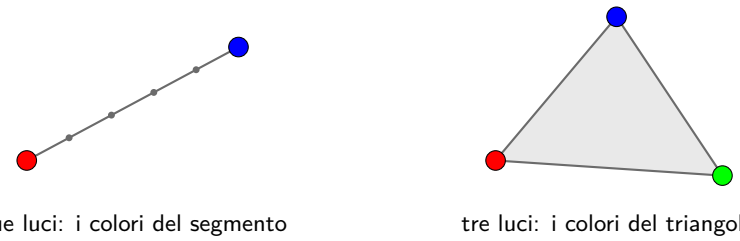


Figura 3.4. In uno spazio colore lineare le miscele di due luci stanno sul segmento che le unisce, quelle di tre luci nel triangolo che definiscono.

Funzioni di color matching. Per calcolare la corrispondenza di qualunque segnale con qualunque terna di primarie si procede in due passi.

1. Si scelgono le primarie $p_1(\lambda)$, $p_2(\lambda)$, $p_3(\lambda)$.
2. Per ogni lunghezza d'onda λ_0, con un passo fissato, si misurano le quantità $c_1(\lambda_0)$, $c_2(\lambda_0)$, $c_3(\lambda_0)$ di ciascuna primaria che servono a riprodurre la luce monocromatica di lunghezza d'onda λ_0.

Le funzioni $c_1(\lambda)$, $c_2(\lambda)$, $c_3(\lambda)$ sono le **funzioni di color matching** (*color matching functions*).

Qualunque segnale spettrale è una combinazione lineare di moltissime luci monocromatiche, cioè una loro somma con dei pesi: il peso di ciascuna è la potenza spettrale a quella lunghezza d'onda. Campionando N lunghezze d'onda $\lambda_1, \dots, \lambda_N$, il segnale diventa un **vettore**, una colonna di N numeri, ognuno la potenza del segnale a una lunghezza d'onda:

$$\vec{t} = \begin{pmatrix} t(\lambda_1) \\ \vdots \\ t(\lambda_N) \end{pmatrix}.$$

La luce monocromatica di lunghezza d'onda λ_i si riproduce con le quantità $c_1(\lambda_i)$, $c_2(\lambda_i)$, $c_3(\lambda_i)$ delle tre primarie. Per le leggi di Grassmann, il segnale $\vec{t}$ si riproduce sommando i contributi di tutte le sue componenti. La quantità della prima primaria, per esempio, è

$$e_1 = c_1(\lambda_1)\, t(\lambda_1) + c_1(\lambda_2)\, t(\lambda_2) + \cdots + c_1(\lambda_N)\, t(\lambda_N),$$

cioè la somma, su tutte le lunghezze d'onda, della quantità di prima primaria che serve per quella luce monocromatica, moltiplicata per quanta di quella luce c'è nel segnale. Allo stesso modo si ottengono e_2 ed e_3. Per scriverlo in breve si mettono le funzioni di color matching nelle righe di una **matrice** C, una tabella di 3 righe e N colonne:

$$C = \begin{pmatrix} c_1(\lambda_1) & \cdots & c_1(\lambda_N) \\ c_2(\lambda_1) & \cdots & c_2(\lambda_N) \\ c_3(\lambda_1) & \cdots & c_3(\lambda_N) \end{pmatrix}, \qquad \vec{e} = C\,\vec{t}. \tag{3.2}$$

Il prodotto $C\,\vec{t}$ di una matrice per un vettore è proprio il calcolo di sopra: ogni componente di $\vec{e}$ si ottiene moltiplicando la riga corrispondente di C per $\vec{t}$, elemento per elemento, e sommando i prodotti. Le componenti e_1, e_2, e_3 di $\vec{e}$ sono quindi le

quantità delle tre primarie che riproducono $\vec{t}$, e ne descrivono il colore. Se un altro segnale $\vec{s}$ corrisponde percettivamente a $\vec{t}$, le stesse e_1, e_2, e_3 riproducono anche $\vec{s}$, sempre per le leggi di Grassmann. In questi termini due metameri sono due spettri diversi che la matrice C porta nello stesso vettore $\vec{e}$.

Esempio (due metameri con i numeri). Con $N = 4$ lunghezze d'onda e numeri inventati, sia

$$C = \begin{pmatrix} 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 \end{pmatrix}:$$

la prima primaria serve per le lunghezze d'onda 1 e 2, la seconda per la 2 e la 3, la terza per la 3 e la 4. Lo spettro $\vec{t}$ ha luce solo nelle lunghezze d'onda 1 e 3, lo spettro $\vec{s}$ solo nella 2 e nella 4: scritti in riga, $\vec{t} = (1, 0, 1, 0)$ e $\vec{s} = (0, 1, 0, 1)$. Ogni riga di C, moltiplicata elemento per elemento per lo spettro, dà dopo la somma:

	per $\vec{t}$	per $\vec{s}$
e_1	$1 \cdot 1 + 1 \cdot 0 + 0 \cdot 1 + 0 \cdot 0 = 1$	$1 \cdot 0 + 1 \cdot 1 + 0 \cdot 0 + 0 \cdot 1 = 1$
e_2	$0 \cdot 1 + 1 \cdot 0 + 1 \cdot 1 + 0 \cdot 0 = 1$	$0 \cdot 0 + 1 \cdot 1 + 1 \cdot 0 + 0 \cdot 1 = 1$
e_3	$0 \cdot 1 + 0 \cdot 0 + 1 \cdot 1 + 1 \cdot 0 = 1$	$0 \cdot 0 + 0 \cdot 1 + 1 \cdot 0 + 1 \cdot 1 = 1$

I due spettri non hanno nessuna lunghezza d'onda in comune, eppure danno lo stesso $\vec{e} = (1, 1, 1)$: sono metameri. Succede perché C riassume 4 numeri con 3, e, come per i coni, un'informazione va persa.

3.6 Spazi colore

RGB. Nello spazio **RGB** le primarie sono luci monocromatiche, per esempio $p_1 = 645{,}2\,\text{nm}$ (rosso), $p_2 = 525{,}3\,\text{nm}$ (verde) e $p_3 = 444{,}4\,\text{nm}$ (blu). Nei monitor le primarie corrispondono ai tre tipi di fosfori.

Per alcune lunghezze d'onda nessuna combinazione positiva delle tre primarie riproduce la luce di prova. Bisogna allora aggiungere una delle primarie alla luce di prova, dall'altra parte dello schermo: è il **matching sottrattivo** (*subtractive matching*), che equivale a un peso negativo. Per questo le funzioni di color matching RGB hanno tratti negativi.

Con le coordinate normalizzate in $[0, 1]$, i colori RGB formano un cubo (Figura 3.5). Il nero sta nell'origine e il bianco nel vertice opposto, $(1, 1, 1)$; sulla diagonale tra i due ci sono i grigi. Tre vertici sono le primarie e gli altri tre le loro somme a due a due: giallo (rosso + verde), magenta (rosso + blu), ciano (verde + blu).

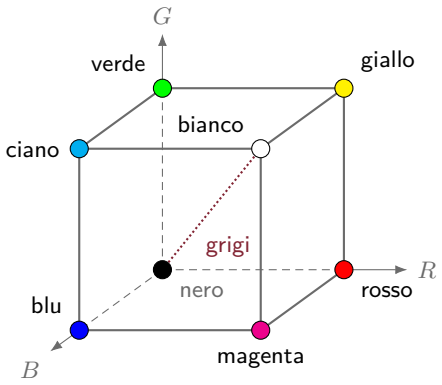


Figura 3.5. Il cubo RGB. Il vertice nero, nell'origine, è nascosto; la diagonale punteggiata contiene i grigi.

CIE XYZ. Lo spazio **CIE XYZ** è stato definito nel 1931 dalla Commissione internazionale per l'illuminazione (CIE, *Commission Internationale de l'Éclairage*). Le sue primarie sono immaginarie, cioè non corrispondono a luci fisiche, ma le funzioni di color matching sono positive ovunque: non serve mai il matching sottrattivo. Le coordinate X, Y, Z di un colore sono le quantità delle tre primarie. Per una visualizzazione in due dimensioni si usano le **coordinate di cromaticità**

$$x = \frac{X}{X+Y+Z}, \qquad y = \frac{Y}{X+Y+Z}, \tag{3.3}$$

che lasciano da parte l'intensità e tengono solo la «qualità» del colore. Con $X = 20$, $Y = 30$, $Z = 50$, per esempio, $x = 20/100 = 0{,}2$ e $y = 30/100 = 0{,}3$. La stessa luce con intensità doppia, $(40, 60, 100)$, dà ancora $x = 40/200 = 0{,}2$ e $y = 0{,}3$: dividendo per la somma, l'intensità si cancella. Nel diagramma di cromaticità (x, y) i colori visibili occupano una regione a forma di ferro di cavallo. I colori monocromatici stanno sul bordo curvo, da 380 a 700 nm, e il bianco verso il centro.

Spazi uniformi. Purtroppo le distanze nel piano (x, y) non rispecchiano le differenze di colore percepite. Le **ellissi di MacAdam** delimitano, attorno a un colore, i colori che non se ne distinguono (*just noticeable differences*). Nel diagramma (x, y) hanno dimensioni molto diverse: piccole nel blu, grandi nel verde. Lo spazio **CIE** $u'v'$ è una trasformazione proiettiva di (x, y) che rende le ellissi più uniformi. La generazione successiva è **CIE $\mathbf{L^*a^*b^*}$**, costruito con formule empiriche: costanti e funzioni scelte perché il risultato si adatti alle misure sull'occhio.

HSV. Lo spazio **HSV** è non lineare e ha dimensioni con un significato percettivo:

- la **tinta** (*hue*, H): quale colore;
- la **saturazione** (*saturation*, S): quanto colore, dal grigio al colore puro;
- il **valore** (*value*, V), o intensità: quanto è luminoso.

Si ottiene mettendo il cubo RGB «sul vertice», cioè guardandolo lungo la diagonale dei grigi, dal bianco verso il nero. I sei vertici colorati formano un esagono attorno

al centro, nell'ordine rosso, giallo, verde, ciano, blu, magenta (Figura 3.6a). La costruzione geometrica procede in quattro passi:

1. si inclina il cubo sul vertice nero, aggiungendo delle cuciture lungo gli spigoli;
2. si schiacciano i sei colori puri su uno stesso piano;
3. si allarga la base;
4. si arrotonda il solido in un cilindro.

Il risultato è un cilindro, o un cono se si fa convergere il fondo nel nero (Figura 3.6b). La tinta è l'angolo attorno all'asse dei grigi: rosso a 0°, verde a 120°, blu a 240°. La saturazione è la distanza dall'asse e il valore la posizione lungo l'asse. Con HSV certe trasformazioni diventano semplici: per cambiare la tinta di un'immagine basta spostare H, per renderne i colori più vivi basta aumentare S.

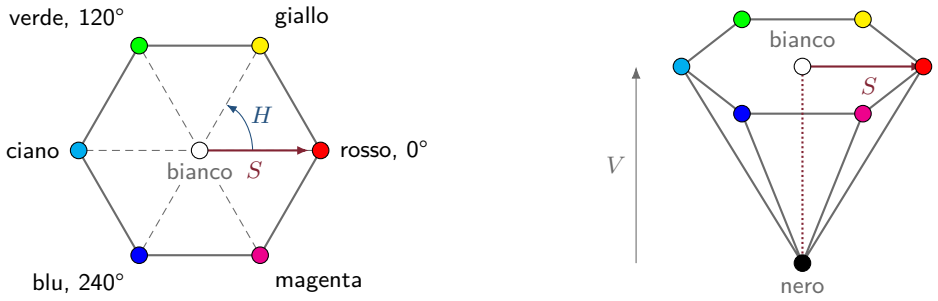


Figura 3.6. Lo spazio HSV. (a) Il cubo RGB visto lungo la diagonale dei grigi: la tinta H è l'angolo a partire dal rosso, la saturazione S la distanza dal centro. (b) L'esacono visto di lato: il valore V cresce dal nero ai colori della base.

3.7 Costanza e adattamento

La percezione del colore non dipende solo dallo spettro che arriva in un punto della retina. I fenomeni che lo mostrano si raccolgono in quattro gruppi:

- la costanza del colore e della chiarezza;
- gli effetti istantanei: il contrasto simultaneo e le bande di Mach;
- gli effetti graduali: l'adattamento alla luce e al buio, l'adattamento cromatico e le post-immagini;
- gli effetti di alto livello, come il dare un nome ai colori (*color naming*).

Definizione (costanza del colore). La **costanza del colore** e della **chiarezza** (*color/lightness constancy*) è la capacità del sistema visivo di percepire le proprietà intrinseche di riflettanza delle superfici nonostante i cambiamenti dell'illuminazione.

In un dipinto di J. S. Sargent (*Le figlie di Edward D. Boit*, 1882) i grembiuli bianchi delle bambine appaiono bianchi sia alla luce sia in ombra. Ancora più netta è

l'illusione della scacchiera di Adelson (*checker-shadow illusion*). Un cilindro proietta un'ombra su una scacchiera, e due caselle hanno nell'immagine esattamente lo stesso grigio: una casella scura in piena luce e una chiara in ombra. Eppure la seconda appare molto più chiara. Se si coprono i dintorni con due strisce di grigio uniforme, l'uguaglianza diventa evidente. Le spiegazioni proposte sono due:

- il contrasto simultaneo;
- la distinzione tra bordi dovuti alla riflettanza (un cambio di materiale) e bordi dovuti all'illuminazione (il contorno di un'ombra): il sistema visivo «sconta» l'ombra.

Nel **contrasto simultaneo** il colore dello sfondo cambia il colore percepito dell'oggetto: due grigi identici sembrano diversi su sfondi diversi (Figura 3.7a). Lo stesso succede con il colore: in un motivo a strisce gialle e blu, quadratini dello stesso rosso appaiono diversi a seconda delle strisce su cui stanno. Nelle **bande di Mach**, al bordo tra due zone uniformi di chiarezza diversa, si percepisce una banda più chiara dal lato chiaro e una più scura dal lato scuro, che fisicamente non esistono (Figura 3.7b).



(a) **(b)**

Figura 3.7. (a) Contrasto simultaneo: i due quadrati centrali hanno lo stesso grigio, ma su sfondo scuro sembra più chiaro. (b) Bande di Mach: ogni fascia è uniforme, ma vicino a ogni bordo sembra più chiara dal lato chiaro e più scura dal lato scuro.

Adattamento cromatico. Il sistema visivo cambia la propria sensibilità in base alle luminanze presenti nel campo visivo, con un meccanismo ancora poco compreso. L'adattamento avviene in due modi.

- Ai diversi livelli di luminosità ci si adatta cambiando l'apertura dell'iride, e quindi la quantità di luce che entra nell'occhio: è quello che succede entrando in un edificio dopo essere stati in pieno sole.
- Alle diverse temperature di colore della luce si adattano le cellule della retina, che cambiano sensibilità. Se, per esempio, la luce rossa aumenta, le cellule sensibili al rosso diminuiscono la loro sensibilità finché la scena non appare di nuovo bianca.

Ci si adatta meglio nelle scene luminose: per questo le scene a lume di candela continuano ad apparire gialle. L'adattamento si può provare. Si fissa per qualche secondo il centro tra due campi colorati, uno ciano e uno giallo, così che le due metà della retina si adattino a colori diversi. Un'immagine guardata subito dopo appare allora colorata in modo diverso nelle due metà.

Noi e il calcolatore. La nostra percezione del colore è diversa da quella di un calcolatore:

- sfruttiamo una visione d'insieme della scena;
- approssimiamo molto;
- non ricaviamo il colore dall'oggetto a cui è associato.

Una prova è la sfida «*name that color*». Si legge una lista di nomi di colori, ognuno stampato con un inchiostro di un altro colore (la parola «verde» in rosso, la parola «rosso» in verde...), e si deve dire il colore dell'inchiostro. Il compito è lento e pieno di errori, perché la lettura della parola interferisce con il riconoscimento del colore.

3.8 Bilanciamento del bianco

Quando guardiamo una foto su uno schermo o stampata, ci adattiamo all'illuminante della stanza, non a quello della scena fotografata. Se il **bilanciamento del bianco** (*white balance*) non è corretto, la foto ha quindi una **dominante** di colore (*color cast*) innaturale.

Con la pellicola si usavano pellicole o filtri diversi per le diverse condizioni di illuminazione. Le fotocamere digitali offrono tre possibilità:

- il bilanciamento automatico;
- le impostazioni per gli illuminanti più comuni: tungsteno, fluorescente, luce del giorno, flash, nuvoloso, ombra, oppure una temperatura di colore in kelvin;
- il bilanciamento personalizzato, con un oggetto di riferimento.

Se nella scena ci sono più illuminanti, il bilanciamento è delicato: punti di riferimento diversi danno risultati diversi. In una foto di un edificio illuminato da luci artificiali sotto un cielo con la luna, per esempio, prendendo come riferimento la luna o la pietra dell'edificio si ottengono due foto molto diverse.

Adattamento di von Kries. Il modello più semplice moltiplica ogni canale per un fattore di guadagno:

$$(R', G', B') = (k_R\, R,\ k_G\, G,\ k_B\, B), \tag{3.4}$$

dove (R, G, B) è il colore registrato, k_R, k_G, k_B sono i guadagni dei tre canali e (R', G', B') è il colore corretto. La sorgente di luce potrebbe avere un effetto più complesso, corrispondente a una matrice 3×3 qualsiasi invece che diagonale.

Il modo migliore per stimare i guadagni è il **cartoncino grigio** (*gray card*). Si fotografa un oggetto neutro, bianco o grigio, e se ne ricava il peso di ogni canale: se l'oggetto è registrato come (r_w, g_w, b_w), si usano i pesi

$$k_R = \frac{1}{r_w}, \qquad k_G = \frac{1}{g_w}, \qquad k_B = \frac{1}{b_w},$$

che riportano l'oggetto neutro a un grigio con i tre canali uguali. Se sotto una luce calda il cartoncino è registrato come (0,8; 0,6; 0,4), per esempio, i pesi sono $k_R = 1{,}25$,

$k_G \approx 1{,}67$ e $k_B = 2{,}5$: il cartoncino diventa $(1, 1, 1)$, e un pixel $(0{,}4;\ 0{,}3;\ 0{,}2)$, con la stessa dominante, diventa il grigio $(0{,}5;\ 0{,}5;\ 0{,}5)$.

Senza un cartoncino grigio bisogna «indovinare» quali pixel corrispondono a oggetti bianchi. Le strategie principali sono nella Tabella 3.2; in alternativa si usano statistiche delle immagini e tecniche di apprendimento.

Ipotesi	Idea e pesi
Mondo grigio (*gray world*)	la media dell'immagine $(r_{ave}, g_{ave}, b_{ave})$ è grigia; si usano i pesi $1/r_{ave}$, $1/g_{ave}$, $1/b_{ave}$
Pixel più luminoso (*brightest pixel*)	le zone di riflesso hanno di solito il colore della sorgente di luce; si usano pesi inversamente proporzionali ai valori dei pixel più luminosi
Gamut mapping	il *gamut* è l'involucro convesso dei colori di tutti i pixel dell'immagine; si cerca la trasformazione che porta il gamut dell'immagine su quello di un'immagine «tipica» sotto luce bianca

Tabella 3.2. Strategie di bilanciamento del bianco senza cartoncino grigio.

Bilanciamento per riconoscimento. Un approccio più recente usa la semantica dell'immagine.

1. Si assegnano i pixel a classi semantiche, come alberi o acqua, in base a forma e posizione.
2. Per ogni classe si calcola l'illuminante che porta il colore medio dei suoi pixel sul colore medio della stessa classe in un database di immagini «tipiche». Ogni classe produce così un'ipotesi di illuminante.
3. Si sceglie l'ipotesi più verosimile dal punto di vista semantico.

3.9 Usi del colore nella computer vision

La Tabella 3.3 raccoglie alcuni usi del colore.

Applicazione	Idea
Indicizzazione e ricerca di immagini	un oggetto o un'immagine si rappresenta con l'istogramma dei suoi colori, e si confrontano gli istogrammi
Rilevamento della pelle	ogni pixel è classificato come pelle o non pelle in base al colore, con modelli statistici
Segmentazione e ricerca	l'immagine si divide in regioni (*blob*) coerenti per colore e texture, e si interroga per regioni (Blobworld)
Calcio robotico (*robot soccer*)	segmentare per colore palla, campo e robot, riducendo la calibrazione manuale dei colori
Inseguimento (*tracking*)	si impara l'aspetto, cioè i colori, delle parti del corpo di ciascuna persona e lo si usa per seguirla
Realismo visivo	la compatibilità tra i colori delle parti di un fotomontaggio dice se l'immagine sembra realistica

Tabella 3.3. Alcuni usi del colore nella computer vision.

3.10 Texture e Local Binary Pattern

Il colore è ben definito; la texture molto meno.

Definizione (texture). La **texture** è una descrizione della disposizione spaziale dei colori o delle intensità in un'immagine, o in una sua regione.

Ci sono due modi di descriverla.

- Nell'**approccio strutturale** la texture è un insieme di **texel** (*texture element*), elementi disposti secondo un motivo regolare o ripetuto, come le figure di una carta da parati. Nelle texture naturali, però, come l'erba o le foglie, è difficile dire quali siano i texel e dove si trovino.
- L'**approccio statistico** nasce proprio da qui. Isolare i texel è difficile o impossibile nelle immagini reali, mentre si possono calcolare quantità numeriche, statistiche, che descrivono la texture a partire dai soli livelli di grigio, o dai colori. È meno intuitivo ma efficiente da calcolare, e serve sia per la classificazione sia per la segmentazione.

Definizione (Local Binary Pattern). Per ogni pixel p si costruisce un numero di 8 bit, $b_1 b_2 \dots b_8$, uno per ciascuno degli 8 vicini, presi in un ordine fisso: $b_i = 0$ se il vicino i ha un valore minore o uguale a quello di p, $b_i = 1$ altrimenti. La texture di un'immagine, o di una regione, si rappresenta con l'**istogramma** di questi numeri: è la misura **LBP** (*Local Binary Pattern*).

Nell'esempio della Figura 3.8 il pixel centrale vale 50 e i vicini, in senso orario dall'angolo in alto a sinistra, valgono 100, 101, 103, 80, 90, 60, 50, 40. I primi sei sono maggiori di 50 e danno 1; il settimo è uguale e l'ottavo minore, e danno

0. Il codice è 11111100 (252 in decimale). Poiché i codici possibili sono $2^8 = 256$, l'istogramma ha 256 classi.

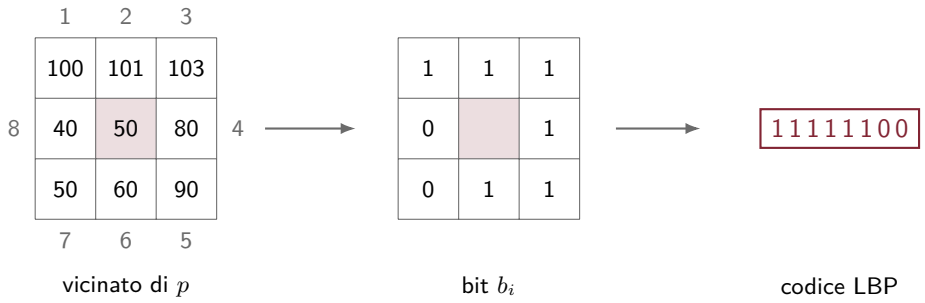


Figura 3.8. Calcolo del codice LBP del pixel centrale. I numeri in grigio indicano l'ordine dei vicini.

Il sistema Fids (*Flexible Image Database System*) recupera da un database le immagini simili a un'immagine data confrontandone gli istogrammi LBP. Con un giardino fiorito come immagine di partenza restituisce altri giardini. Con un edificio, invece, trova anche immagini diverse per contenuto: le misure di basso livello non sempre trovano immagini simili per significato. L'LBP è stato usato, fra l'altro, per classificare gli alberi decidui, per individuare il cancro nei Pap test e per cercare regioni di interesse nelle biopsie del seno e del melanoma.

3.11 L'immagine in memoria

In C un'immagine si può rappresentare con questa struttura:

```
typedef struct {
    int h, w, c;
    float *data;
} image;
```

dove:

- `h` è l'altezza e `w` la larghezza, in pixel;
- `c` è il numero di canali: 3 per un'immagine RGB, 1 per una in scala di grigi;
- `data` è un array di `float` con valori in $[0, 1]$.

La matrice tridimensionale dell'immagine (righe, colonne, canali) è linearizzata in un array monodimensionale. Un modo comune è mettere i canali uno dopo l'altro, ciascuno riga per riga: il valore del pixel (x, y) nel canale z sta allora in posizione $z \cdot h \cdot w + y \cdot w + x$. Per arrivarci si saltano z canali interi, di $h \cdot w$ valori ciascuno, poi y righe intere, di w valori ciascuna, poi x pixel. Con $h = 2$, $w = 3$ e $c = 3$, per esempio, ogni canale occupa 6 posizioni e ogni riga 3, e il pixel $(1, 1)$ del canale 2 sta in posizione $2 \cdot 6 + 1 \cdot 3 + 1 = 16$.